Scaffold FYP — Research Gap Analysis & Recommended Direction

The Landscape: What Already Exists (and What Doesn't)

After surveying ~70 recent papers (2024–2026) across LLM-based tutoring, knowledge tracing, IDE telemetry, hint generation, and educational guardrails, here is the state of the art and where the genuine gap lives.

Existing Systems and Their Blind Spots

LLM Tutoring Tools for Programming

CodeHelp (Liffiton et al., Koli Calling 2023) — The first widely cited LLM wrapper for CS education. Uses prompt-based guardrails to prevent GPT from giving direct code. Students enter a question + code + optional error message and get a conceptual response. **Limitation**: entirely reactive (student must ask), no student modeling whatsoever, no awareness of what concepts the student has or hasn't mastered. Guardrails are prompt-level only and have been shown to fail under adversarial rephrasing.

CodeAid (Kazemitabaar et al., CHI 2024) — A step up from CodeHelp.

Offers multiple input templates (explain code, fix error, help write), generates pseudocode with line-by-line explanations instead of raw code, and annotates student code with inline suggestions. Deployed with 700+ students over a semester. **Limitation**: still reactive, still no student modeling, no knowledge tracing, no awareness of student's concept mastery. Treats every student identically regardless of skill level.

STAP (Xie et al., ISAIE 2025) — Socratic Tutor for Adaptive Programming. Formalizes key concepts: Socratic hints, Minimum Viable Hint (MVH), and answer leakage (defined as disclosure of syntactically complete runnable code). Uses a four-stage pipeline — Checking, Correcting, Complementing, Segmenting — to analyze code before generating hints. **Limitation**: no student model, no classroom study, only illustrative walkthroughs, future work explicitly calls for "authentic student data" and "classroom studies." No knowledge tracing integration.

ClassAid (Zhang et al., CHI 2026) — The closest existing system to what Scaffold envisions. Real-time classroom orchestration with TA Agents that provide personalized support. Instructor dashboard lets the instructor switch between Technical Mode (direct coding solutions), Heuristic Mode (hint-based guidance), and Auto Mode (system decides). Deployed with 54 students. **Limitation**: the critical decision of *what mode to use* is made by the **instructor manually** watching a dashboard, not driven by an automated student model. Auto Mode selects between technical/heuristic but does not use knowledge tracing — it uses simple heuristics like inactivity detection. No formal knowledge tracing, no concept mastery modeling, no cross-session student profiles.

Closing the Loop (Phung et al., SIGCSE TS 2026) — An instructor-in-the-loop system. AI generates three types of hints (planning, debugging, optimization). When AI hints are unhelpful (22% of cases), students can escalate to a human instructor. **Limitation**: no automated struggle detection — relies entirely on the student manually requesting help. No knowledge tracing. No proactive intervention.

Guardrail and Safety Systems

CodeGuard (Raihan et al., EACL Findings 2026) — A guardrail framework specifically for CS education. Introduces a taxonomy for classifying prompts (Irrelevant, Relevant-Safe, Relevant-Unsafe), the CodeGuard dataset (8,000 prompts), and PromptShield, a lightweight sentence-encoder model that achieves 0.93 F1 for detecting unsafe prompts. Reduces policy-violating code completions by 30–65%. **Limitation**: operates only at the prompt classification level. Not integrated with any tutoring system, no hint generation, no student modeling.

EduGuardBench (Jiang et al., AAAI 2026) — A benchmark for evaluating pedagogical fidelity and adversarial safety of LLMs as simulated teachers. Calibrates an LLM judge against 200 human labels across 14 models. **Limitation**: a benchmark, not a system. Evaluates models in isolation, not within a deployed tutoring context.

Scaffolding Collapse (July 2026) — Introduces the concept of "scaffolding collapse," where Socratic tutors degrade under adversarial pressure and start giving direct answers. Proposes representation alignment as a mitigation. **Limitation**: studied only in the math domain, not programming. Programming leakage is fundamentally different (partial code, pseudocode, function signatures create a spectrum of disclosure that binary "leaked/not-leaked" doesn't capture).

Answer Leakage Robustness (Zhao et al., ACL 2026) — Defines six categories of adversarial student attacks against LLM tutors and evaluates tutor robustness. Finds that simple prompting-based defenses can reduce leakage. **Limitation**: evaluated on math tutoring tasks, not programming. The code domain introduces unique challenges — is showing a function signature "leakage"? Is pseudocode "leakage"? Is showing 3 of 10 lines "leakage"?

Knowledge Tracing for Programming

Code-DKT (Shi et al., EDM 2022) — Extends Deep Knowledge Tracing with code2vec representations of student submissions. Predicts next-submission correctness. Foundational work for code-aware KT. **Limitation**: offline only — runs over historical submission logs, not integrated with any real-time feedback system.

srcML-DKT (EDM 2025) — Enhances DKT using srcML-based code representations. Uses ASTs for richer structural features. Evaluated on 610 students across 4 academic years. **Limitation**: same as Code-DKT — an offline prediction model, not connected to any intervention system.

DPKT (Nature Scientific Reports, 2025) — Difficulty-aware Programming Knowledge Tracing via LLMs. Uses LLMs to extract text understanding difficulty and knowledge concept difficulty. **Limitation**: focused on KT model accuracy, not on using the KT output to drive adaptive feedback.

Open-Ended Knowledge Tracing (OKT) (Liu et al., EMNLP 2022) — The only work that combines generative LLMs with an underlying KT model. Injects the knowledge state into a generative LLM to predict student open-ended responses (code). **Limitation**: used to *predict* what code a student would write, not to *generate hints* for a student. The KT-to-generation connection exists but has never been used for pedagogical intervention.

Detecting Struggling Programmers (Schwartz et al., August 2025) — Uses proficiency taxonomies (Bloom's Taxonomy) combined with coding history to detect struggling students. Extends prior KT models by combining coding proficiencies with complete coding history. **Limitation**: detection is still offline/batch, not real-time. No connection to an intervention system.

IDE Telemetry and Monitoring

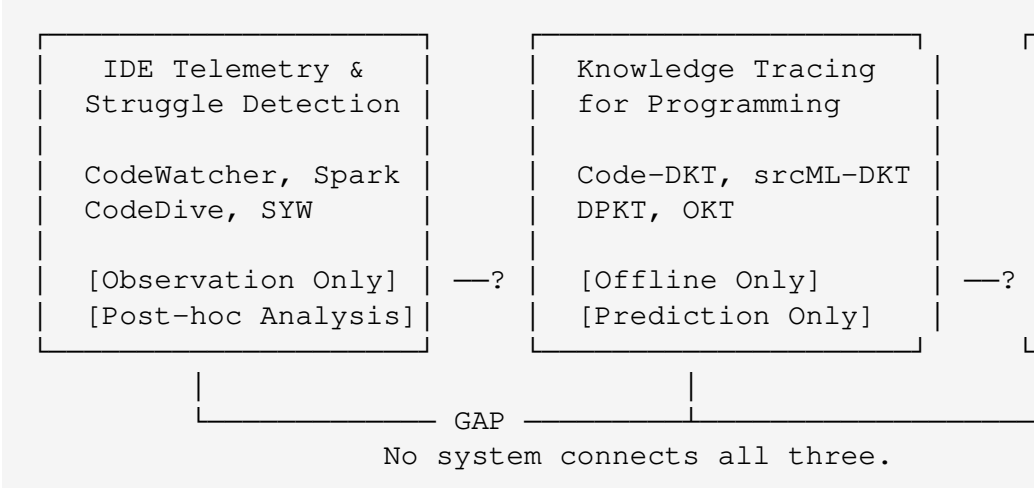
CodeWatcher (ICSME 2025) — VS Code plugin that captures fine-grained interaction events: CGT insertions, deletions, copy-paste actions, focus shifts. Uses a Python API + MongoDB backend. Designed for understanding how developers interact with code generation tools. **Limitation**: observation-only — collects data for post-hoc research analysis, not for real-time intervention.

Spark (Jan 2026) — Real-time monitoring of multi-faceted programming exercises. Instructor-facing dashboard showing student progress with code inspection. Designed for the instructor to identify struggling students. **Limitation**: still a visualization tool for instructors, not an automated intervention system.

CodeDive (Park et al., 2025) — Web-based IDE with real-time code activity monitoring for programming education. **Limitation**: similar to Spark — monitoring and visualization, not automated adaptive feedback.

The Research Gap: What No One Has Done

After mapping all of the above, the gap becomes clear. These three areas each have deep research, but **no one has connected them into a closed loop**:



Specifically:

1. **Knowledge tracing models exist but don't drive hint generation.** Code-DKT can tell you a student struggles with recursion, but no system uses that signal to make the LLM hint about base cases instead of loop invariants.

2. **Hint generation systems exist but are student-model-blind.** STAP and CodeAid generate hints from the code alone. They don't know this student mastered arrays last week but has never correctly written a recursive function. They give the same hint to a strong student and a weak one.

3. **Telemetry systems exist but are observation-only.** CodeWatcher and Spark collect rich behavioral data but feed it to dashboards for human inspection, not to automated intervention pipelines.

4. **ClassAid partially closes the loop but relies on the instructor as the decision-maker.** The instructor manually switches modes. This doesn't scale — an instructor can't dynamically adjust 40 students' hint levels simultaneously during a live lab. The system needs to do this automatically using a student model.

5. **Answer leakage for code is undefined at fine granularity.** STAP defines leakage as "syntactically complete runnable code." But programming has a rich spectrum: variable name suggestions, function signatures, pseudocode with gaps, partial implementations, full solutions with one line removed. No one has formalized a graduated leakage taxonomy for code or studied which level of disclosure optimizes learning.

6. **No system is proactive.** Every existing system waits for the student to ask for help. But research on struggle detection shows that struggling students are often the *least likely* to seek help. A system that detects struggle from IDE telemetry and proactively offers calibrated support would be novel.

Recommended FYP Direction

Title (Working)

"Scaffold: Knowledge-State-Driven Adaptive Scaffolding with Proactive Struggle Detection for Programming Labs"

Research Question

"Does knowledge-state-conditioned, proactively triggered LLM

scaffolding improve programming learning outcomes compared to reactive, student-model-blind LLM assistance in a live lab setting?"

What Makes This a Research Project, Not Just Engineering

The 50% research component answers questions that the literature has not answered:

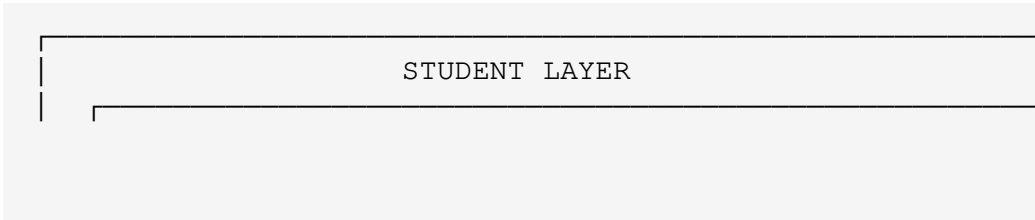
RQ1 — Student Modeling: How accurately can a lightweight knowledge tracing model, updated in real-time from IDE telemetry (not just submission verdicts), predict which concept a student is struggling with? Compare Code-DKT on submission logs vs. an enriched model that also ingests telemetry signals (edit frequency, compilation errors, idle time, copy-paste patterns).

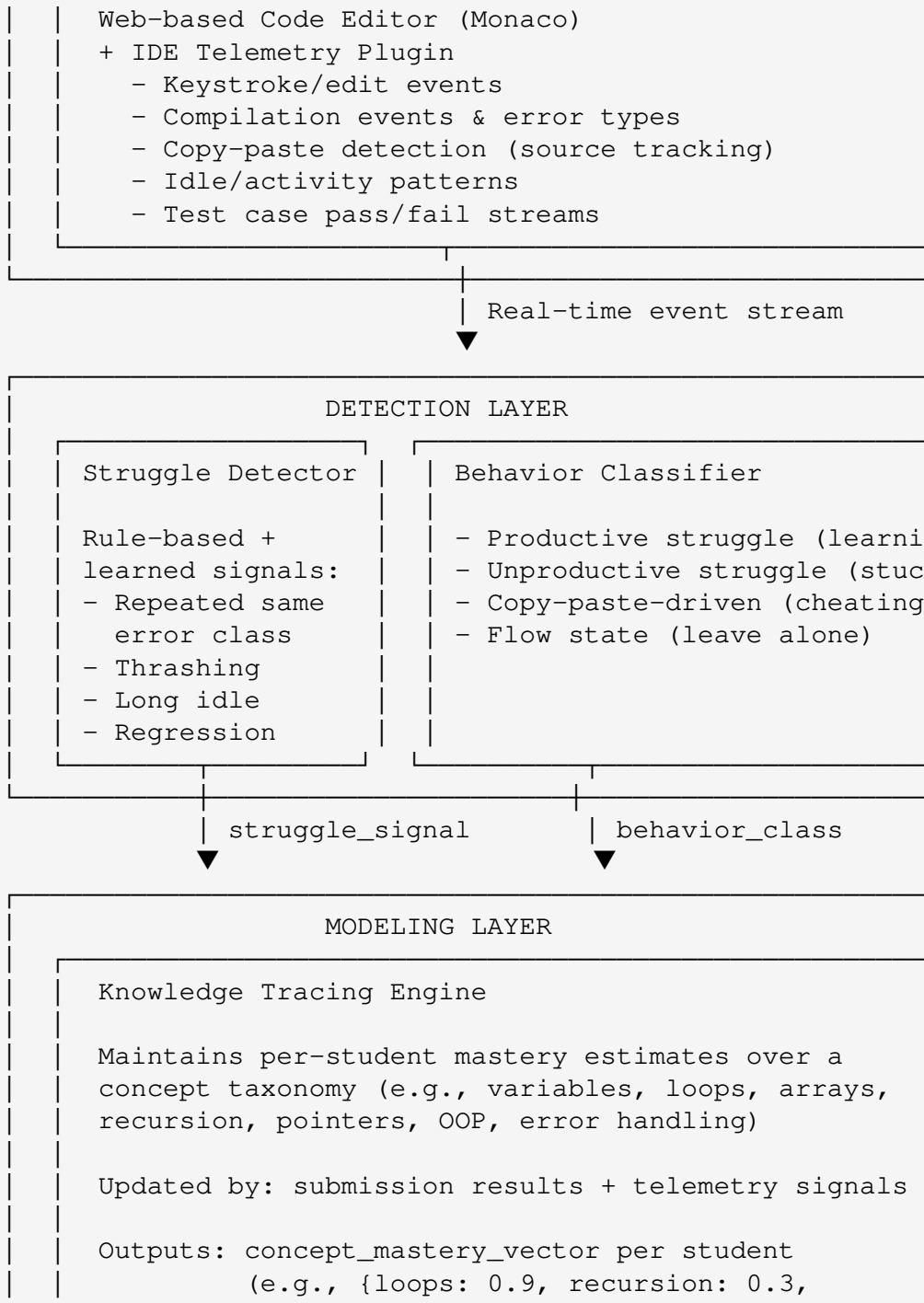
RQ2 — KT-Conditioned Hints vs. Generic Hints: Does injecting the student's knowledge state into the LLM's context (e.g., "This student has mastered loops but has not demonstrated understanding of pointer arithmetic") produce hints that lead to better learning outcomes (measured by: time to correct solution, success on a subsequent transfer problem without hints, self-reported understanding)?

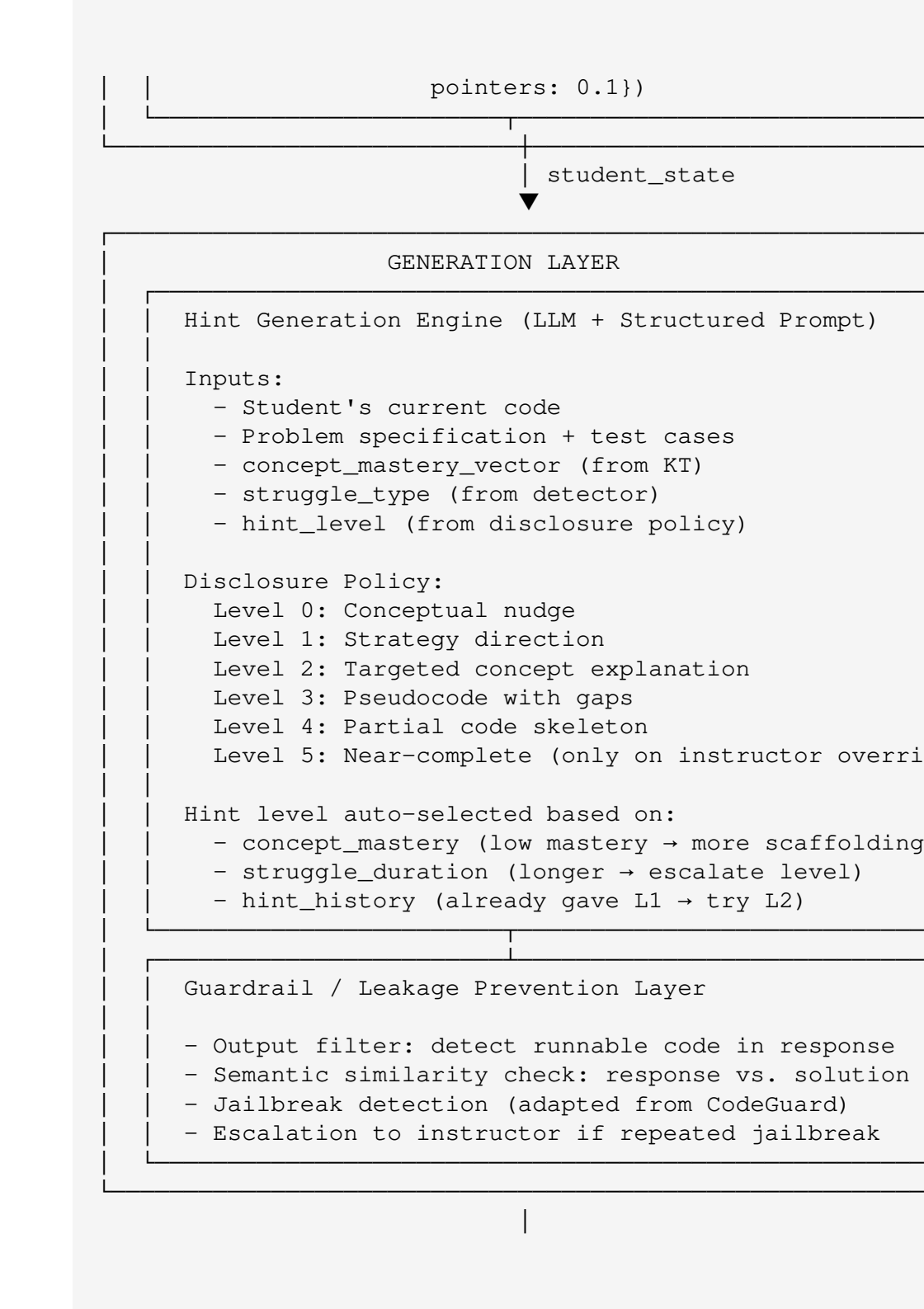
RQ3 — Proactive vs. Reactive Intervention: Does proactively offering a hint when the system detects struggle (via telemetry signals) lead to better outcomes than waiting for the student to ask? Or does it cause learned helplessness / over-reliance?

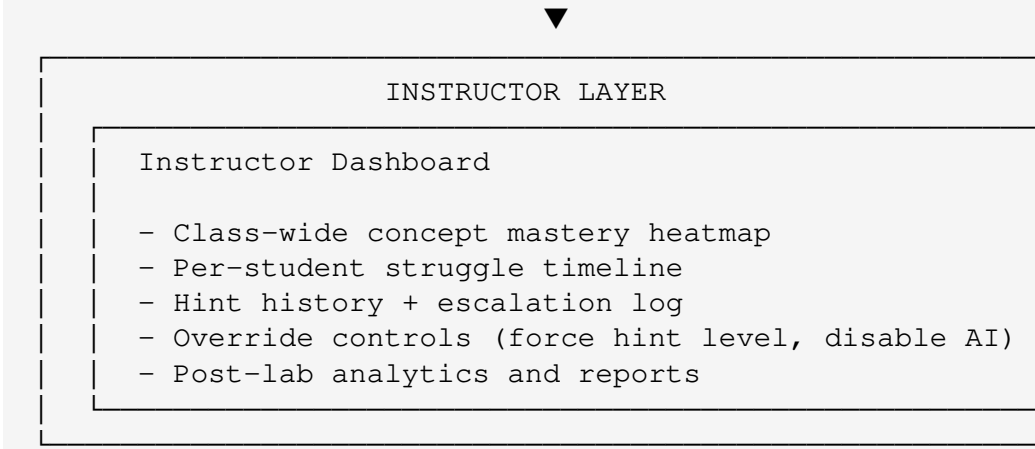
RQ4 — Code Leakage Spectrum: What level of code disclosure (on a defined graduated scale from conceptual hint → pseudocode → partial code → full solution) maximizes learning while minimizing frustration? This requires defining and validating a Code Disclosure Taxonomy.

System Architecture (50% Development)




Web-based Code Editor (Monaco)
+ IDE Telemetry Plugin
- Keystroke/edit events
- Compilation events & error types
- Copy-paste detection (source tracking)
- Idle/activity patterns
- Test case pass/fail streams
Real-time event stream
DETECTION LAYER
Struggle Detector
Rule-based +
learned signals:
- Repeated same
error class
- Thrashing
- Long idle
- Regression
Behavior Classifier
- Productive struggle (learni
- Unproductive struggle (stuc
- Copy-paste-driven (cheating
- Flow state (leave alone)
struggle_signal
behavior_class
MODELING LAYER
Knowledge Tracing Engine
Maintains per-student mastery estimates over a
concept taxonomy (e.g., variables, loops, arrays,
recursion, pointers, OOP, error handling)
Updated by: submission results + telemetry signals
Outputs: concept_mastery_vector per student
(e.g., {loops: 0.9, recursion: 0.3,


pointers: 0.1})
student_state
GENERATION LAYER
Hint Generation Engine (LLM + Structured Prompt)
Inputs:
- Student's current code
- Problem specification + test cases
- concept_mastery_vector (from KT)
- struggle_type (from detector)
- hint_level (from disclosure policy)
Disclosure Policy:
Level 0: Conceptual nudge
Level 1: Strategy direction
Level 2: Targeted concept explanation
Level 3: Pseudocode with gaps
Level 4: Partial code skeleton
Level 5: Near-complete (only on instructor overri
Hint level auto-selected based on:
- concept_mastery (low mastery → more scaffolding
- struggle_duration (longer → escalate level)
- hint_history (already gave L1 → try L2)
Guardrail / Leakage Prevention Layer
- Output filter: detect runnable code in response
- Semantic similarity check: response vs. solution
- Jailbreak detection (adapted from CodeGuard)
- Escalation to instructor if repeated jailbreak



Differentiation from Existing Systems

Dimension	CodeHelp	CodeAid	STAP	ClassAid	Scaffold
Student model	None	None	None	None	Knowledg tracing
Intervention trigger	Student asks	Student asks	Student asks	Student asks or instructor switches	Auto (struggle detection) + student can ask
Hint calibration	One-size-fits-all	One-size-fits-all	Socratic only	Instructor chooses mode	Auto based on KT state + struggle type
Code leakage prevention	Prompt guardrail	Pseudocode-only output	Formalized MVH	Mode-based	Graduate disclosure taxonomy + output filter

Proactive support	No	No	No	Inactivity detection only	Yes (telemetry driven)
Classroom deployment	Yes (52 students)	Yes (700+)	No	Yes (54 students)	Yes (target: 30–60 in FAST lab
Cross-session student profile	No	No	No	No	Yes (KT persists across labs)

Scope for a 1-Year FYP

Semester 1 (Research-Heavy)

- Literature review and formal proposal (Months 1–2)
- Define concept taxonomy for one course (e.g., Data Structures in C++) (Month 2)
- Build the IDE telemetry plugin (web-based Monaco editor with event capture) (Months 2–3)
- Implement and compare 2–3 knowledge tracing approaches on historical data (if available from your department's grading systems, or from public datasets like the CSEDM competition data) (Months 3–5)
- Pilot the struggle detector with rule-based signals on a small group (Month 5–6)

Semester 2 (Development + Evaluation)

- Integrate KT output into LLM hint generation via structured prompts (Month 7)
- Build the guardrail/leakage prevention layer (Month 7–8)
- Build the instructor dashboard (Month 8–9)
- Classroom study: deploy in a real FAST NUCES lab with 30–60 students (Month 9–10)
    - Control group: students use a standard CodeHelp-style reactive assistant

    - Treatment group: students use Scaffold with full KT-driven adaptive scaffolding
    - Measure: time to solution, code quality, success on a subsequent unseen problem, hint usage patterns, student satisfaction survey
- Analysis and thesis writing (Months 10–12)

Where the Research Contribution Lives

Your contribution is NOT "we built an AI chatbot for programming." Your contribution is one or more of these empirical findings:

1. **Telemetry-enriched knowledge tracing outperforms submission-only KT for programming tasks.** If you show that incorporating edit patterns, compilation events, and idle times into Code-DKT improves prediction accuracy, that's a publishable result (venue: EDM, L@S).

2. **KT-conditioned hints produce measurably better learning outcomes than generic hints.** If your classroom study shows that students who receive hints calibrated to their knowledge state perform better on transfer tasks, that validates the approach (venue: SIGCSE, CHI, AIED).

3. **Proactive struggle-triggered intervention helps without causing over-reliance.** If you can show that proactive hints help struggling students without reducing strong students' independence, that answers a genuine open question (the Closing the Loop paper explicitly flags this as unknown).

4. **A graduated code disclosure taxonomy with empirical validation.** If you define, implement, and measure which level of code disclosure (conceptual → pseudocode → partial code) optimizes the learning/frustration tradeoff, that's a conceptual contribution (builds directly on STAP's MVH formalization).

Key Papers to Cite

These form the backbone of your related work section:

- CodeHelp: Liffiton et al. (Koli Calling 2023)
- CodeAid: Kazemitabaar et al. (CHI 2024)
- STAP: Xie et al. (ISAIE 2025)
- ClassAid: Zhang et al. (CHI 2026)
- Closing the Loop: Phung et al. (SIGCSE TS 2026)
- CodeGuard: Raihan et al. (EACL Findings 2026)
- Code-DKT: Shi et al. (EDM 2022)
- srcML-DKT: (EDM 2025)
- Open-Ended KT: Liu et al. (EMNLP 2022)
- Detecting Struggling Programmers: Schwartz et al. (Aug 2025)
- CodeWatcher: (ICSME 2025)
- Answer Leakage Robustness: Zhao et al. (ACL 2026)
- Scaffolding Collapse: (July 2026)
- Hint Generation Survey: "Navigating the Landscape of Hint Generation Research" (2024)
- DPKT: Difficulty-aware Programming KT via LLMs (Nature Scientific Reports, 2025)
- Adaptive vs Planned Metacognitive Scaffolding: (ScienceDirect, 2025)

Addressing FYP Failure Criteria

"Idea not feasible" — Every individual component exists and has been validated independently. CodeWatcher proves telemetry collection works. Code-DKT proves knowledge tracing on code works. STAP proves LLM scaffolding works. Your contribution is integrating them into a closed loop and empirically testing whether the loop improves outcomes. That's feasible within a year.

"No substantial programming effort" — You're building: a web-based code editor with real-time telemetry; a streaming event pipeline; a knowledge tracing engine; an LLM hint generation service with structured prompting; an output filtering guardrail layer; and an instructor dashboard. That's 5+ interconnected services, not a weekend wrapper.

"No learning curve" — Your team will need to understand: knowledge tracing theory (BKT/DKT), educational data mining, LLM prompt

engineering for pedagogical applications, real-time event processing, experimental study design with human subjects, and educational measurement (how to validly measure "learning"). None of this is in your standard CS curriculum.

"A similar FYP has already been done" — Search your department's FYP repository. You'll likely find generic "AI tutors." None of them will have knowledge tracing, telemetry-based struggle detection, graduated code disclosure policies, or a controlled classroom study. The specificity of the research questions and the closed-loop architecture differentiate Scaffold from any wrapper-around-ChatGPT project.